Отчёт по лабораторной работе №5

Построение графиков на языке Julia

Базлов В.А.

Содержание

Цель работы

Задание

Выполнение работы

1. Повторение примеров раздела 5.2

1.1. Основные графики и ряд Тейлора

1.2. Точечные графики и аппроксимация

1.3. Две оси, полярные и параметрические графики

1.4. Поверхности, контуры и векторное поле

1.5. Анимация, гипоциклоида, погрешности и распределения

1.6. Подграфики и типы рядов

2. Самостоятельная работа — раздел 5.4

2.1. Типы графиков и стили линий

2.2. Функция $\pi x^2 \ln(x)$

2.3. Функция $x^3 - 3x$

2.4. Две функции и две оси ординат

2.5. Экспериментальные и случайные данные

2.6. Анимация синусоиды

2.7. Гипоциклоиды и эпициклоиды

Выводы

Цель работы

Целью лабораторной работы является освоение средств построения и оформления графиков на языке Julia с использованием библиотеки Plots.jl. В ходе работы изучаются различные виды двумерной и трёхмерной визуализации, параметрические кривые, поверхности, графики с погрешностями, гистограммы, несколько осей координат и анимация.

Задание

1. Используя JupyterLab, повторить примеры из раздела 5.2, дополнив графики обозначениями осей координат, легендами, названиями траекторий и заголовками.

2. Выполнить все задания для самостоятельной работы из раздела 5.4: построить различные типы графиков, оформить линии и оси, визуализировать экспериментальные и случайные данные, а также создать анимации.

Выполнение работы

Работа выполнена в JupyterLab на языке Julia. Для построения графиков используется пакет Plots.jl, для генерации случайных данных — Random, для статистических распределений — Distributions, а для вычисления средних значений — Statistics. Ячейки ноутбука выполняются последовательно.

Примечание к иллюстрациям: изображения оформлены в виде реконструированных ячеек JupyterLab с соответствующими фрагментами кода Julia и результатами построения графиков из ноутбука. Это не необработанные снимки экрана работающего интерфейса; внешний вид воспроизведён для единообразного оформления отчёта.

1. Повторение примеров раздела 5.2

1.1. Основные графики и ряд Тейлора

Сначала построена простая кривая функции, заданной аналитическим выражением. Для наглядности добавлены заголовок, подписи горизонтальной и вертикальной осей, легенда и координатная сетка.

[6]:
```
f(x) = (3x^2 + 6x - 9) * exp(-0.3x)
x = collect(-5:0.1:10)
y = f.(x)

plot(x, y, title="Простая кривая", xlabel="x", ylabel="f(x)",
     label="(3x² + 6x − 9)e⁻⁰·³ˣ", color=:blue, grid=true)
```

[6]:

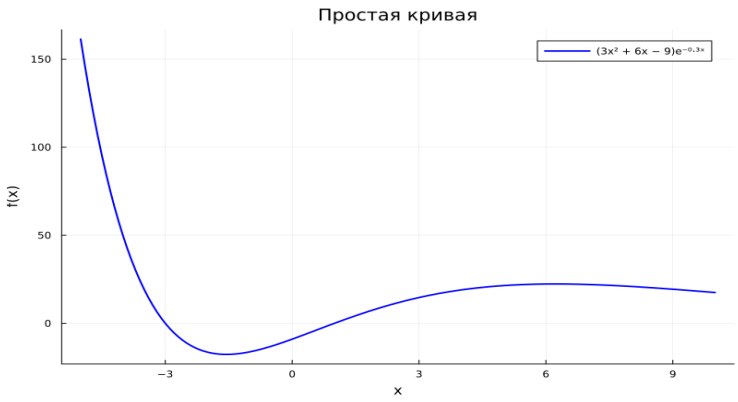


Рисунок 1 — График простой функции

Далее построены графики синуса и его приближения рядом Тейлора. На одном рисунке показаны точная функция и приближённое значение, что позволяет сравнить их поведение.

[9]:
```
plot(xx, sin_taylor.(xx), label="Ряд Тейлора",
     title="Оформление графика ряда Тейлора", xlabel="x", ylabel="y",
     color=:blue, linewidth=3, fillrange=0, fillalpha=0.15,
     background_color=:ivory, xticks=-5:1:5, yticks=-1:0.5:1,
     grid=true)
```

[9]:



Рисунок 2 — Сравнение sin(x) и ряда Тейлора

1.2. Точечные графики и аппроксимация

Для демонстрации точечной визуализации сгенерированы случайные координаты. Сначала данные отображены на плоскости, затем — в трёхмерном пространстве.

[11]:

```
Random.seed!(10)
n = 50
xs, ys = rand(n), rand(n)
ms = 3 .+ 10 .* rand(n)
scatter(xs, ys, markersize=ms, markerstrokewidth=0,
        title="Случайные точки на плоскости",
        xlabel="X", ylabel="Y", label="Наблюдения", legend=:topleft)
```

[11]:

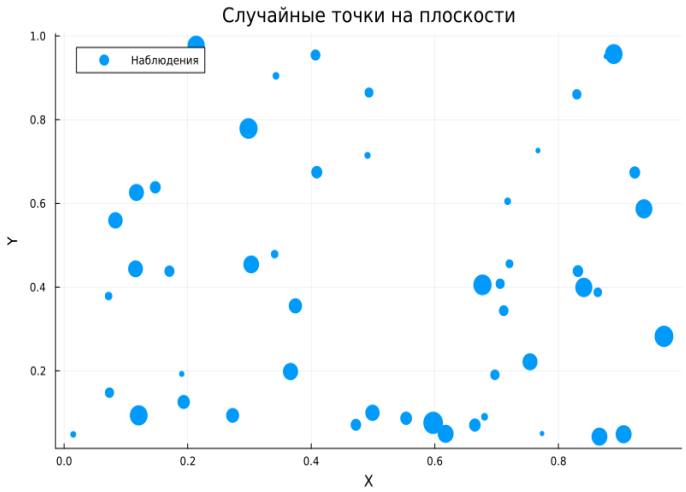


Рисунок 3 — Точечный график случайных данных в 2D

[12]:

```
zs = rand(n)
scatter(xs, ys, zs, markersize=ms, markerstrokewidth=0,
        title="Случайные точки в пространстве",
        xlabel="X", ylabel="Y", zlabel="Z", label="Наблюдения")
```

[12]:

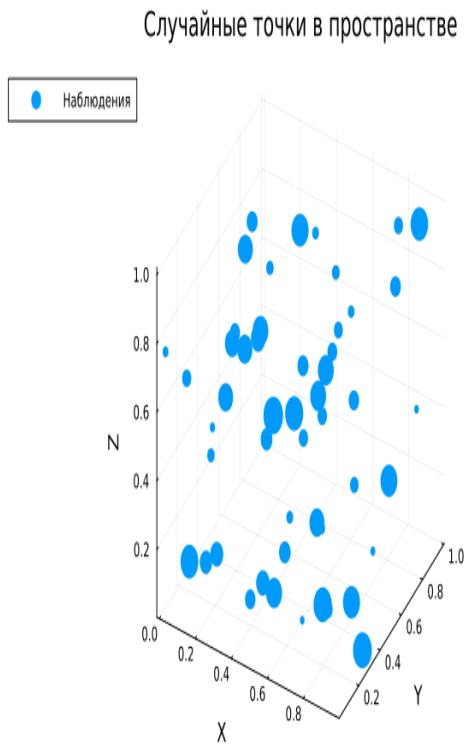


Рисунок 4 — Точечный график случайных данных в 3D

На следующем графике показаны исходные данные и аппроксимирующий полином пятой степени. Аппроксимация позволяет представить общий характер зависимости.

[14]:
```
Random.seed!(11)
xa = collect(0:0.01:9.99)
ya = exp.(1 .+ xa) .+ 4000 .* randn(length(xa))
A = hcat(ones(length(xa)), xa, xa.^2, xa.^3, xa.^4, xa.^5)
c = A \ ya
fit = A * c

p = scatter(xa, ya, markersize=2, alpha=0.45, label="Исходные данные",
            title="Аппроксимация полиномом 5-й степени",
            xlabel="x", ylabel="y")
plot!(p, xa, fit, linewidth=3, color=:red, label="Полином 5-й степени")
display(p)
```

[14]:

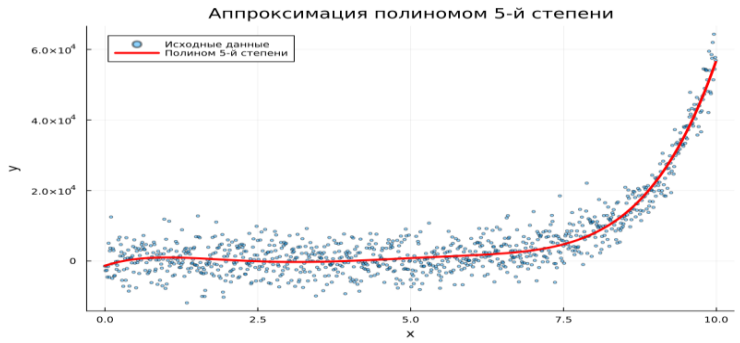


Рисунок 5 — Аппроксимация данных полиномом пятой степени

1.3. Две оси, полярные и параметрические графики

Для одновременного отображения двух последовательностей с разными масштабами использованы две оси ординат. Это позволяет сохранить читаемость обеих зависимостей.

[16]:
```
Random.seed!(12)
t = 1:100
y1 = cumsum(randn(100))
y2 = 10 .* cumsum(randn(100))
p = plot(t, y1, label="Первая траектория", color=:blue,
         xlabel="Номер измерения", ylabel="y1", title="Две оси ординат",
         grid=true)
plot!(twinx(), t, y2, label="Вторая траектория", color=:red,
      ylabel="y2", grid=false)
display(p)
```

[16]:



Рисунок 6 — Графики с двумя осями ординат

Полярная кривая представлена в декартовых координатах через параметрические выражения $X=r(\theta)\cos(\theta)$ и $Y=r(\theta)\sin(\theta)$.

[18]:

```
r(θ) = 1 + cos(θ) * sin(θ)^2
θ = collect(range(0, 2π, length=500))
rp = r.(θ)
plot(rp .* cos.(θ), rp .* sin.(θ), aspect_ratio=:equal,
     title="Полярная кривая r(θ)=1+cos(θ)sin²(θ)",
     xlabel="X", ylabel="Y", label="Кривая", linewidth=2)
```

[18]:

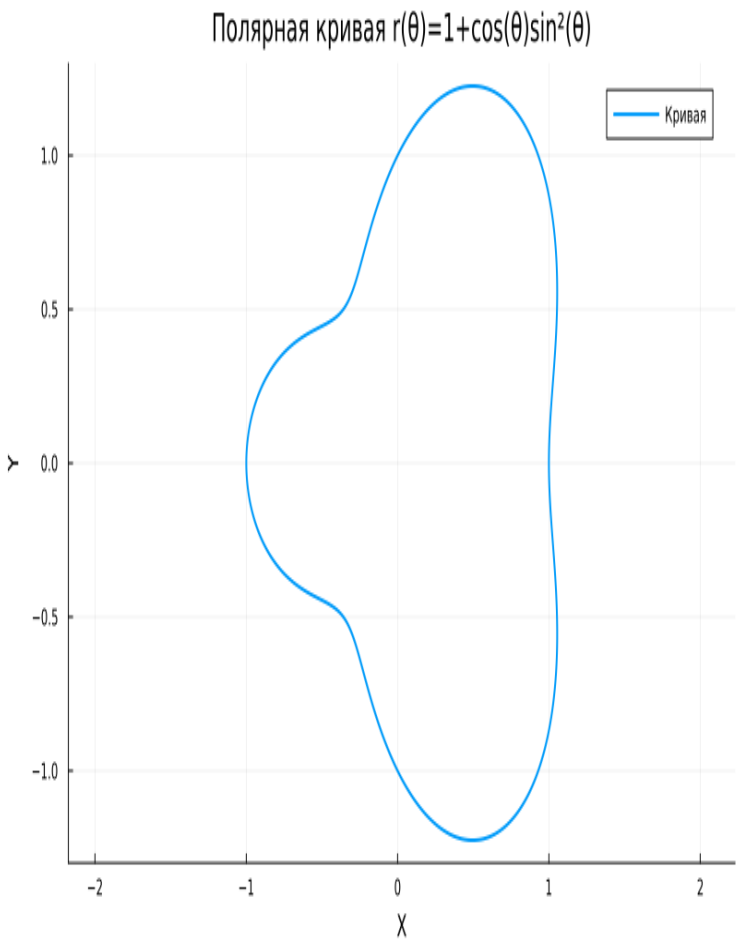


Рисунок 7 — Полярная кривая

Параметрическая кривая на плоскости задана двумя зависимостями от параметра t. Отдельно построена параметрическая поверхность тора в трёхмерном пространстве.

[20]:

```
t = collect(range(0, 2π, length=500))
plot(sin.(t), sin.(2 .* t), aspect_ratio=:equal,
     title="Параметрическая кривая на плоскости",
     xlabel="x(t)", ylabel="y(t)", label="x=sin(t), y=sin(2t)")
```

[20]:

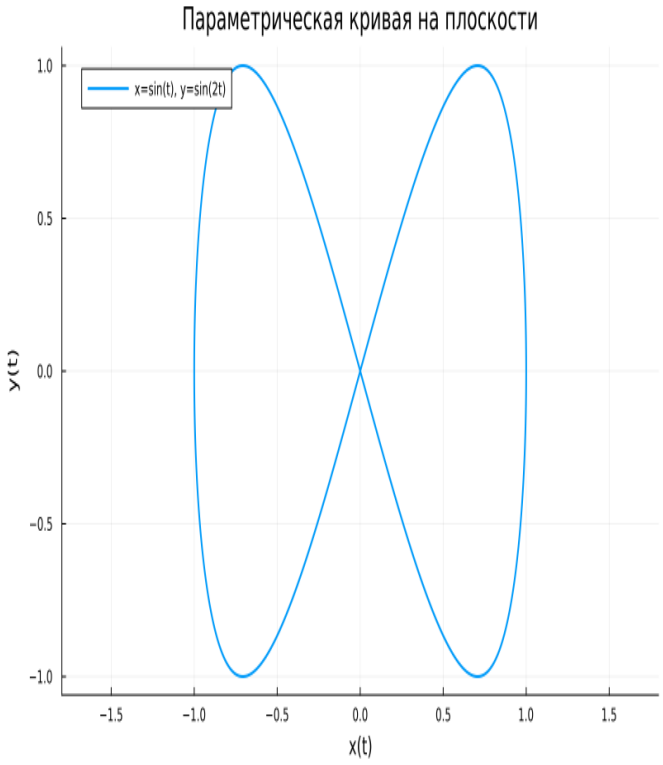


Рисунок 8 — Параметрическая кривая на плоскости

[21]:

```
u = collect(range(0, 2π, length=100))
v = collect(range(0, 2π, length=100))
X = [cos(a) * (2 + cos(b)) for a in u, b in v]
Y = [sin(a) * (2 + cos(b)) for a in u, b in v]
Z = [sin(b) for a in u, b in v]
surface(X, Y, Z, title="Параметрическая поверхность – тор",
        xlabel="X", ylabel="Y", zlabel="Z", label="Поверхность")
```

[21]:

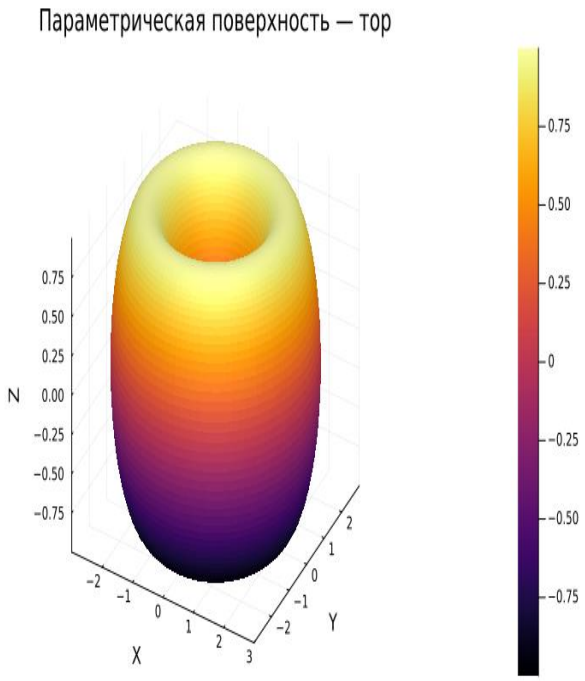


Рисунок 9 — Параметрическая поверхность — тор

1.4. Поверхности, контуры и векторное поле

Для функции двух переменных $z=x^2-y^2$ построена поверхность, её каркасное представление и линии уровня. Эти способы визуализации показывают одну и ту же зависимость с разных сторон.

[23]:

```
xg = collect(range(-2, 2, length=80))
yg = collect(range(-2, 2, length=80))
g(x, y) = x^2 - y^2
Zg = [g(a, b) for a in xg, b in yg]
surface(xg, yg, Zg, title="Поверхность z=x²−y²",
        xlabel="x", ylabel="y", zlabel="z", label="z=f(x,y)")
```

[23]:

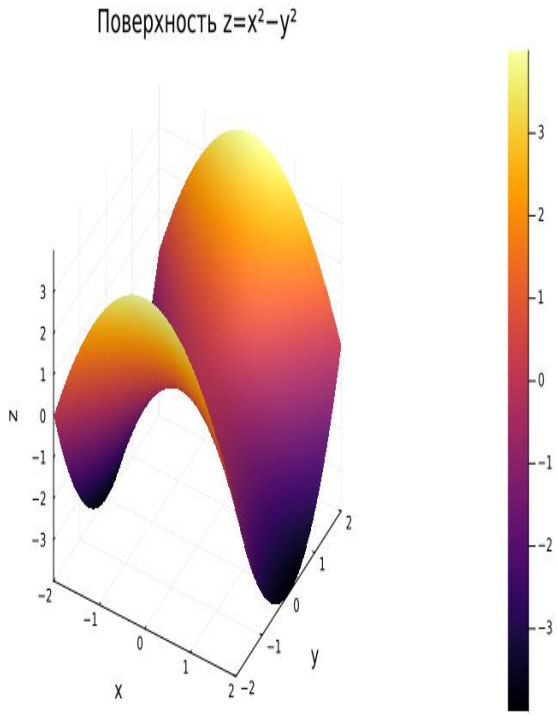


Рисунок 10 — Поверхность z=x²−y²

[24]:
```
wireframe(xg, yg, Zg, title="Каркасная поверхность",
          xlabel="x", ylabel="y", zlabel="z", label="Каркас")
```

[24]:

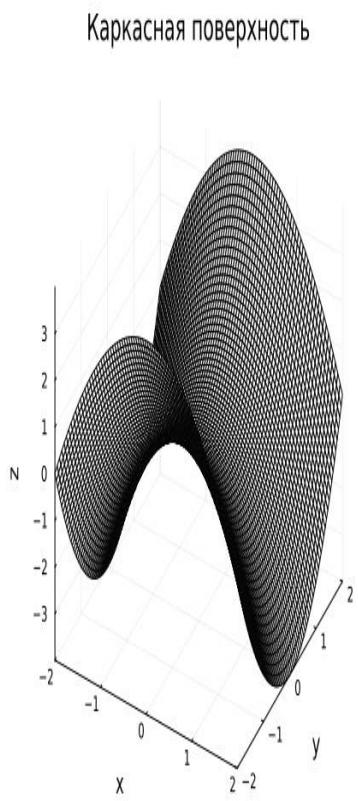


Рисунок 11 — Каркасное представление поверхности

[25]:
```
contour(xg, yg, Zg, title="Линии уровня z=x²−y²",
        xlabel="x", ylabel="y", colorbar_title="z", label="Уровни")
```

[25]:

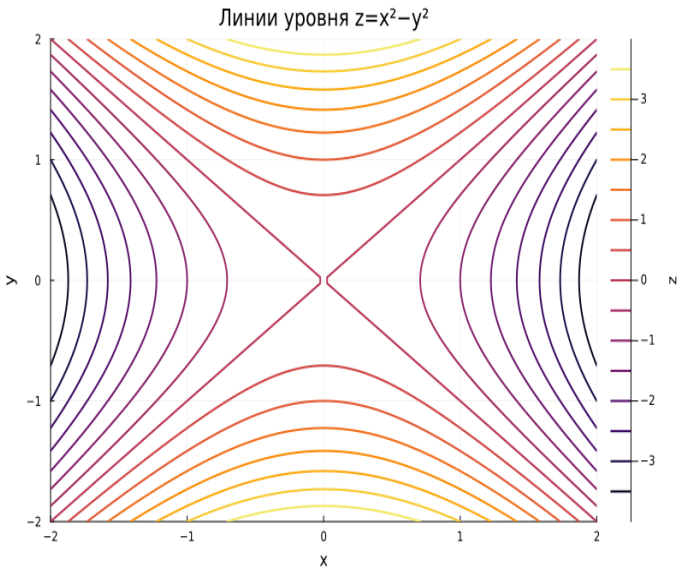


Рисунок 12 — Контурный график функции двух переменных

Также построено векторное поле градиента функции двух переменных. Направление и длина стрелок показывают направление и относительную величину изменения функции.

[26]:

```
h(x, y) = x^3 - 3x + y^2
xv = collect(range(-2, 2, length=17))
yv = collect(range(-2, 2, length=17))
Xv = [a for a in xv, b in yv]
Yv = [b for a in xv, b in yv]
U = (3 .* Xv.^2 .- 3) ./ 25
V = (2 .* Yv) ./ 25
quiver(vec(Xv), vec(Yv), quiver=(vec(U), vec(V)),
       title="Векторное поле градиента h(x,y)",
       xlabel="x", ylabel="y", label="Векторы",
       xlims=(-2,2), ylims=(-2,2), aspect_ratio=:equal)
```

[26]:

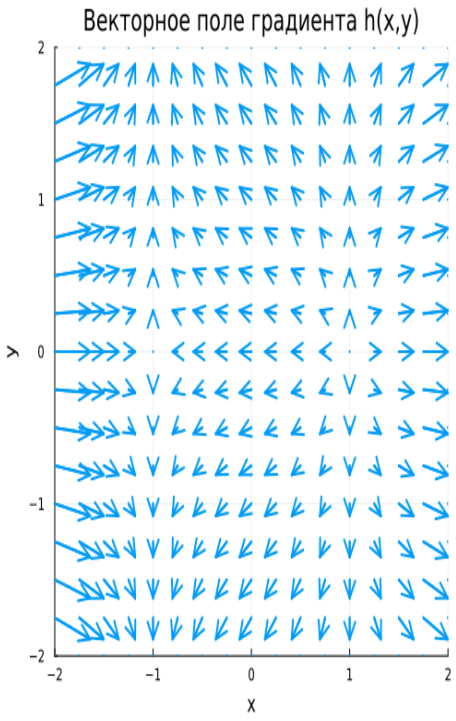


Рисунок 13 — Векторное поле

1.5. Анимация, гипоциклоида, погрешности и распределения

Для демонстрации анимации поверхности сохранён один характерный кадр. В ноутбуке последовательность кадров объединяется в GIF-файл.

[28]:

```
Xsurf = collect(range(-5, 5, length=35))
Ysurf = collect(range(-5, 5, length=35))
anim_surface = @animate for phase in range(0, 2π, length=40)
    Zsurf = [sin(x + 10sin(phase)) + cos(y) for x in Xsurf, y in Ysurf]
    surface(Xsurf, Ysurf, Zsurf, zlims=(-2,2),
            title="Анимация поверхности", xlabel="x", ylabel="y", zlabel="z",
            legend=false)
end
gif(anim_surface, "surface_animation.gif", fps=12)
```

[28]:

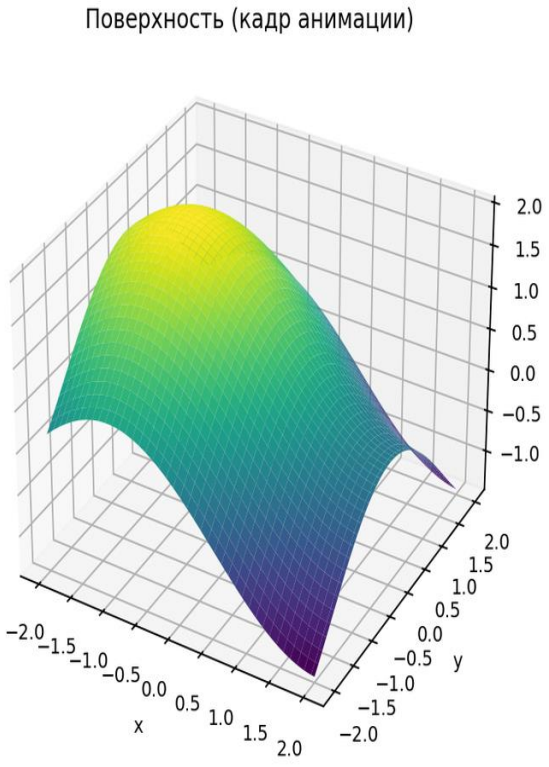


Рисунок 14 — Пример кадра анимации поверхности

Построена гипоциклоида — траектория точки окружности, катящейся внутри другой окружности. В самостоятельных заданиях далее рассмотрены несколько значений параметра k.

[30]:

```
function hypocycloid(k; r=1.0, points=800)
    q = denominator(rationalize(k))
    t = range(0, 2π*q, length=points)
    x = r*(k-1).*cos.(t) .+ r.*cos.((k-1).*t)
    y = r*(k-1).*sin.(t) .- r.*sin.((k-1).*t)
    return x, y
end
xh, yh = hypocycloid(3)
plot(xh, yh, aspect_ratio=:equal, title="Гипоциклоида, k=3",
     xlabel="x", ylabel="y", label="Гипоциклоида")
```

[30]:

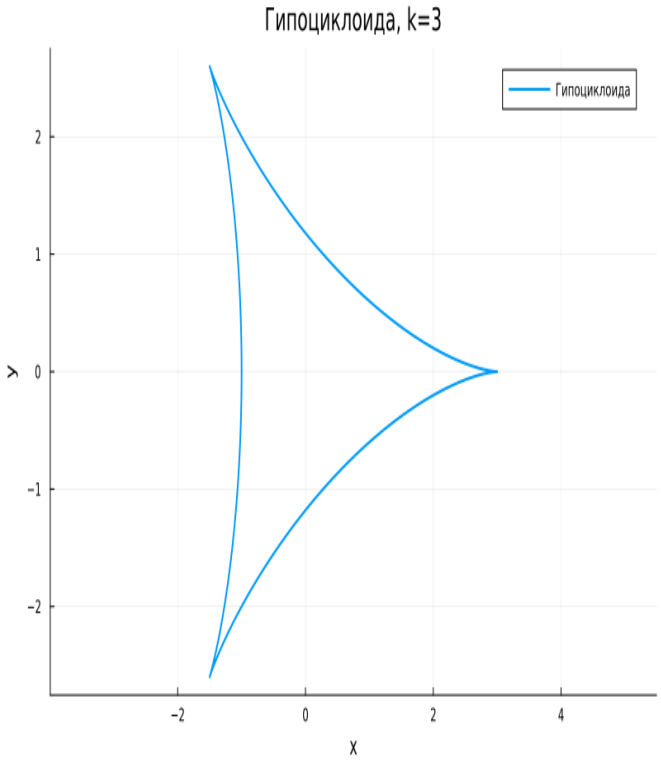


Рисунок 15 — Пример гипоциклоиды

Для данных с погрешностью использованы вертикальные интервалы ошибок. Гистограмма показывает распределение выборки, а график нормального распределения сравнивает выборочные данные с теоретической плотностью.

[32]:

```
Random.seed!(13)
sds = [1, 1/2, 1/4, 1/8, 1/16, 1/32]
nobs = 10
ym = [mean(sd .* randn(nobs)) for sd in sds]
errs = 1.96 .* sds ./ sqrt(nobs)
plot(1:length(ym), ym, yerror=errs, marker=:circle,
     title="Измерения с доверительными интервалами",
     xlabel="Номер измерения", ylabel="Среднее значение",
     label="Среднее ± 1.96·SE")
```

[32]:

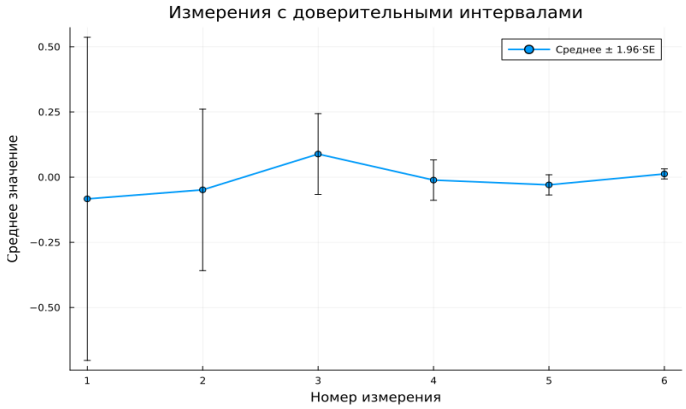


Рисунок 16 — График измерений с погрешностями

[34]:

```
ages = rand(15:55, 1000)
histogram(ages, bins=20, title="Гистограмма случайного возраста",
          xlabel="Возраст, лет", ylabel="Количество", label="Наблюдения")
```

[34]:

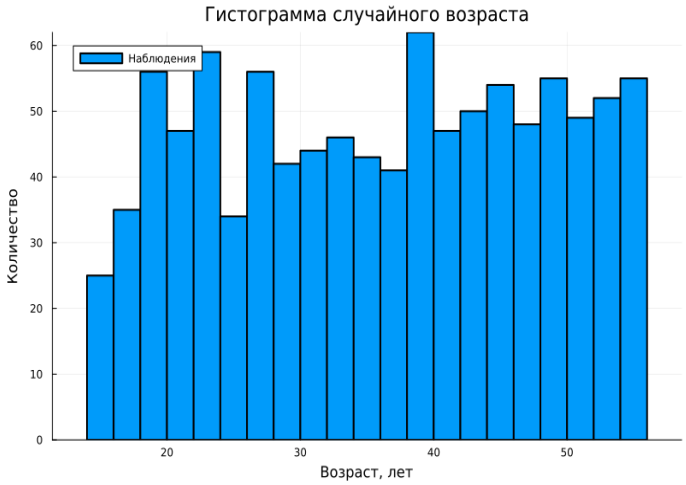


Рисунок 17 — Гистограмма случайного возраста

[35]:
```
d = Normal(35.0, 10.0)
ages_normal = rand(d, 1000)
histogram(ages_normal, bins=30, normalize=:pdf,
          title="Нормальное распределение", xlabel="Возраст, лет",
          ylabel="Плотность вероятности", label="Выборка")
plot!(collect(5:0.1:65), pdf.(d, collect(5:0.1:65)),
      linewidth=2, label="Теоретическая плотность")
```

[35]:

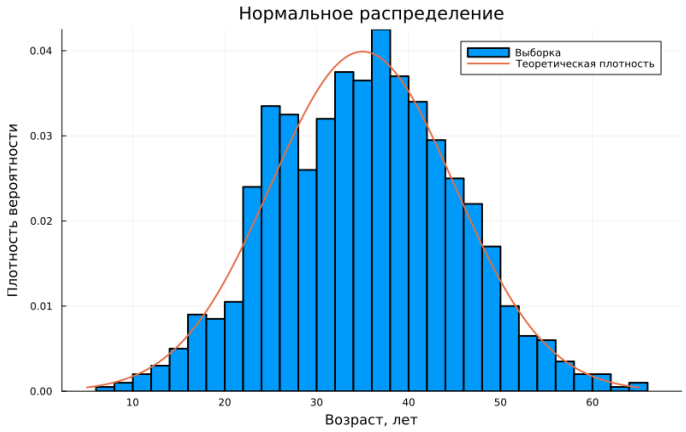


Рисунок 18 — Нормальное распределение

1.6. Подграфики и типы рядов

В одном графическом окне размещены четыре подграфика. Дополнительно показаны распространённые типы рядов: линия, точки, столбцы, ступенчатый график и другие варианты.

[37]:
```
xsub = collect(range(-2, 2, length=30))
ysub = rand(30, 4)
plot(xsub, ysub, layout=(2,2), title=["Ряд 1" "Ряд 2" "Ряд 3" "Ряд 4"],
     xlabel="x", ylabel="y", label=["A" "B" "C" "D"], size=(850,600))
```

[37]:

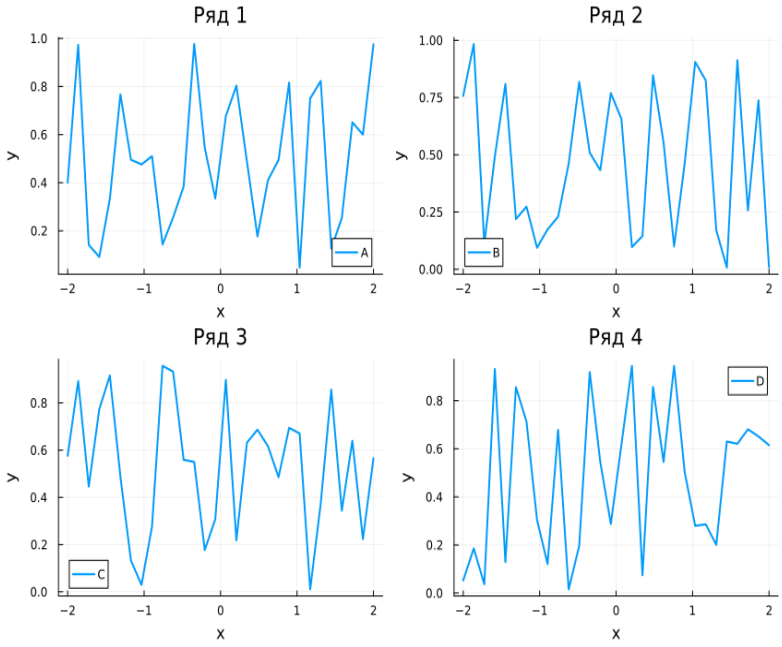


Рисунок 19 — Пример расположения нескольких графиков в одном окне

[38]:
```
series_types = [:line, :scatter, :bar, :steppre, :sticks, :path]
titles = ["line", "scatter", "bar", "steppre", "sticks", "path"]
plot(rand(20), seriestype=series_types, layout=(2,3),
     title=titles, legend=false, xlabel="Индекс", ylabel="Значение",
     size=(900,600))
```

[38]:

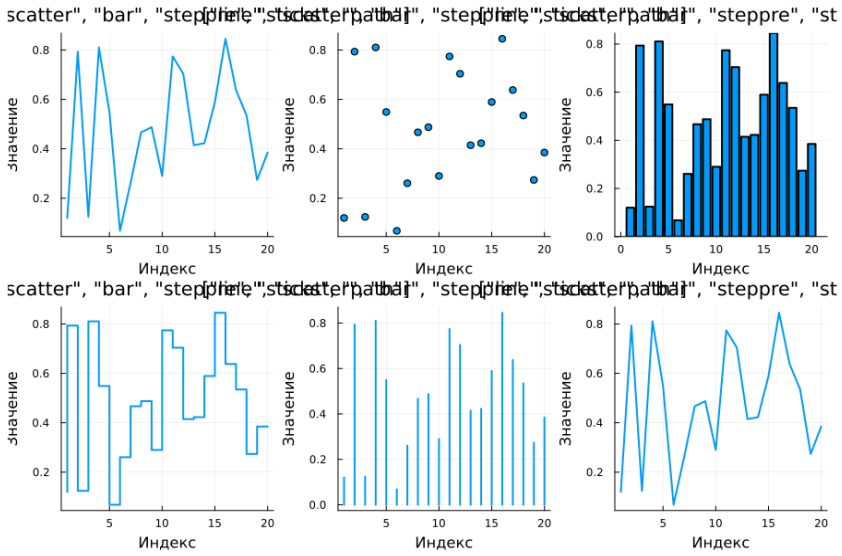


Рисунок 20 — Различные типы графических рядов

2. Самостоятельная работа — раздел 5.4

2.1. Типы графиков и стили линий

В первом задании функция y=sin(x) построена несколькими способами в одном графическом окне: линейный график, точки, столбцы, ступенчатый график, гистограмма значений и полярное представление.

[40]:

```
x = collect(range(0, 2π, length=40))
y = sin.(x)
p1 = plot(x, y, title="Линейный", label="sin(x)", xlabel="x", ylabel="y")
p2 = scatter(x, y, title="Точечный", label="sin(x)", xlabel="x", ylabel="y")
p3 = bar(x[1:4:end], y[1:4:end], title="Столбчатый", label="sin(x)", xlabel="x", ylabel="y")
p4 = plot(x, y, seriestype=:steppost, title="Ступенчатый", label="sin(x)", xlabel="x", ylabel="y")
p5 = histogram(y, bins=12, title="Гистограмма значений", label="sin(x)", xlabel="y", ylabel="Частота")
p6 = plot(y .* cos.(x), y .* sin.(x), aspect_ratio=:equal,
          title="Полярное представление", label="sin(θ)", xlabel="X", ylabel="Y")
plot(p1,p2,p3,p4,p5,p6, layout=(3,2), size=(1000,900),
     plot_title="Разные типы графиков функции y=sin(x)")
```

[40]:

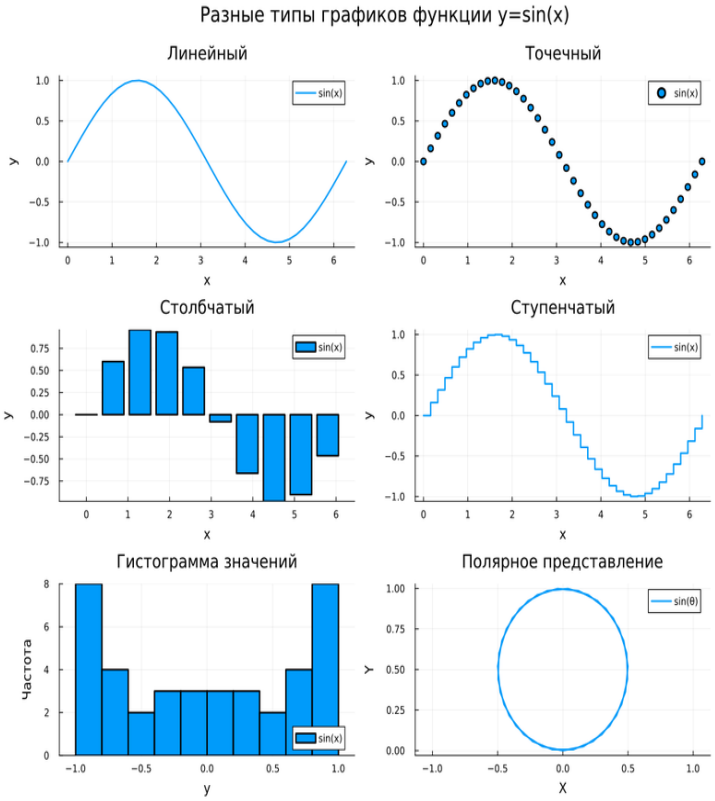


Рисунок 21 — Различные типы графиков функции y=sin(x)

Во втором задании одна и та же функция отображена с разными стилями линий. Для каждой линии добавлена запись в легенде.

[42]:
```
x = collect(range(0, 2π, length=300))
styles = [:solid, :dash, :dot, :dashdot, :dashdotdot]
p = plot(title="Стили линий для y=sin(x)", xlabel="x", ylabel="sin(x)")
for (i, st) in enumerate(styles)
    plot!(p, x, sin.(x), linestyle=st, linewidth=2,
          label=string(st), linealpha=1 - 0.12*(i-1))
end
display(p)
```

[42]:

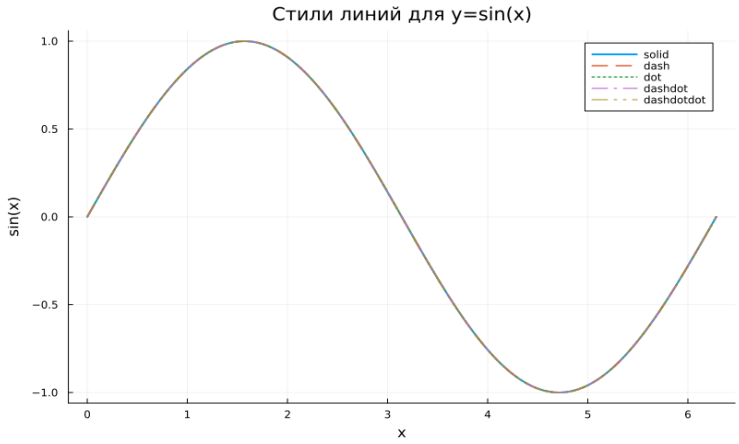


Рисунок 22 — Стили линий графика y=sin(x)

2.2. Функция πx²ln(x)

Построен график функции y(x)=πx²ln(x) на положительном промежутке аргумента. График окрашен в красный цвет, рамка осей — в зелёный. Добавлены подписи, деления оси x и заголовок.

[44]:
```
x3 = collect(range(0.1, 4, length=400))
y3 = π .* x3.^2 .* log.(x3)
plot(x3, y3, color=:red, linewidth=2.5,
     title="y(x) = πx²ln(x)", xlabel="x", ylabel="y(x)",
     legend=false, grid=true, xticks=0:0.5:4,
     guidefont=font(13, "DejaVu Sans"),
     tickfont=font(10, "DejaVu Sans"),
     titlefont=font(14, "DejaVu Sans"),
     foreground_color_border=:green,
     left_margin=8Plots.mm, bottom_margin=8Plots.mm,
     size=(850,550))
```

[44]:

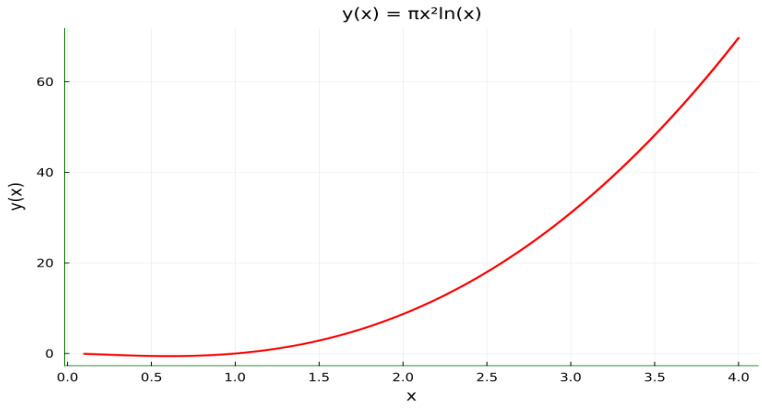


Рисунок 23 — График функции y(x)=πx²ln(x)

2.3. Функция x^3-3x

Для заданного вектора x=(-2,-1,0,1,2) вычислены значения функции $y=x^3-3x$. Результат показан четырьмя способами: отдельные точки, линии, линии с маркерами и гладкая кривая. Изображение сохраняется в PNG-файл.

[46]:
```
x4 = [-2, -1, 0, 1, 2]
y4 = x4.^3 .- 3 .* x4
p1 = scatter(x4, y4, title="Точки", label="Значения", xlabel="x", ylabel="y")
p2 = plot(x4, y4, title="Линии", label="Ломаная", xlabel="x", ylabel="y", marker=:none)
p3 = plot(x4, y4, title="Линии и точки", label="Значения", xlabel="x", ylabel="y", marker=:circle)
xdense = collect(range(-2, 2, length=300))
ydense = xdense.^3 .- 3 .* xdense
p4 = plot(xdense, ydense, title="Кривая", label="y=x³−3x", xlabel="x", ylabel="y")
fig4 = plot(p1,p2,p3,p4, layout=(2,2), size=(900,700),
            plot_title="Графики функции y=x³−3x")
savefig(fig4, "figure_familiya.png")
display(fig4)
println("Сохранено: figure_familiya.png")
```

[46]:

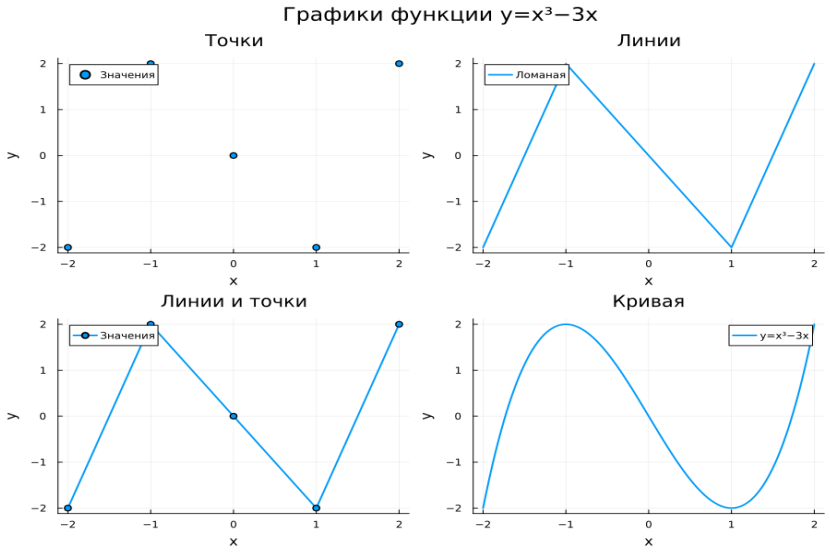


Рисунок 24 — Четыре представления функции $y=x^3-3x$

2.4. Две функции и две оси ординат

На интервале x=3…6 построены функции $y_1=\pi x$ и $y_2=\exp(x)\cos(x)$. При использовании общей оси ординат масштабы функций сильно различаются, поэтому одна из зависимостей может выглядеть визуально менее заметной. Для устранения этого недостатка построен второй вариант с двумя осями Y.

[48]:
```
x5 = collect(3:0.1:6)
y51 = π .* x5
y52 = exp.(x5) .* cos.(x5)
p = plot(x5, y51, color=:blue, label="y₁=πx",
         title="Две функции на одной оси Y",
         xlabel="x", ylabel="y", grid=true)
plot!(p, x5, y52, color=:red, label="y₂=e^cos(x)")
display(p)
```

[48]:

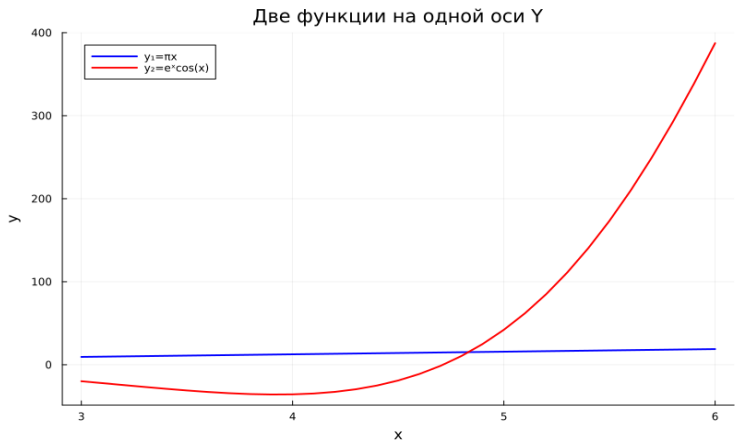


Рисунок 25 — Две функции на одной оси ординат

[49]:
```
p = plot(x5, y51, color=:blue, label="y₁=πx",
         title="Две функции с двумя осями Y",
         xlabel="x", ylabel="y₁=πx", grid=true)
plot!(twinx(), x5, y52, color=:red, label="y₂=e^cos(x)",
      ylabel="y₂=e^cos(x)", grid=false)
display(p)
```

[49]:

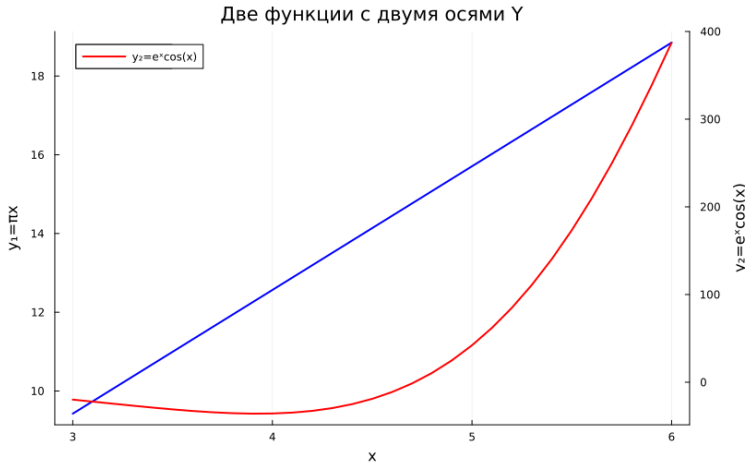


Рисунок 26 — Две функции с двумя осями ординат

2.5. Экспериментальные и случайные данные

Для шестого задания созданы условные экспериментальные измерения. Погрешность каждого измерения показана вертикальным отрезком с засечками.

[51]:

```
x6 = 1:8
Random.seed!(56)
true_values = 2 .* x6 .+ 1
measurement_error = [0.25,0.35,0.30,0.45,0.40,0.50,0.35,0.45]
measured = true_values .+ (rand(length(x6)) .- 0.5) .* 2 .* measurement_error
plot(x6, measured, yerror=measurement_error, marker=:circle,
     linestyle=:dash, color=:purple, label="Измерения ± погрешность",
     title="Экспериментальные данные", xlabel="Номер опыта",
     ylabel="Измеренное значение", grid=true)
```

[51]:

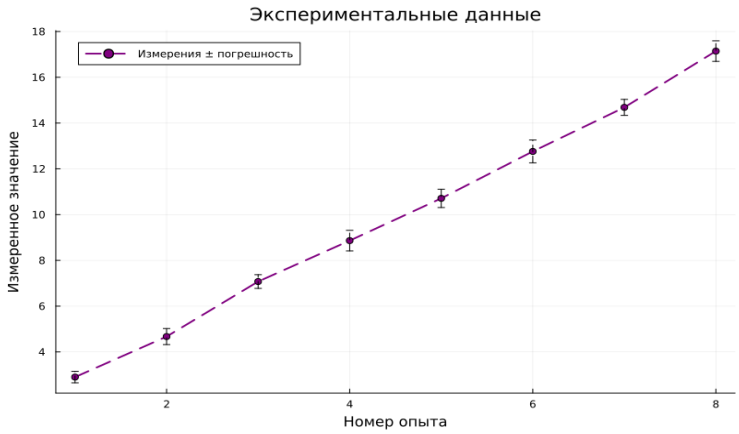


Рисунок 27 — Экспериментальные данные с погрешностью

В седьмом и восьмом заданиях сгенерированы случайные координаты. Данные представлены соответственно в виде двумерного и трёхмерного точечных графика.

[53]:

```
Random.seed!(7)
x7, y7 = rand(60), rand(60)
scatter(x7, y7, label="Случайные наблюдения",
        title="Случайные данные (2D)", xlabel="X", ylabel="Y",
        markersize=5, markerstrokewidth=0)
```

[53]:

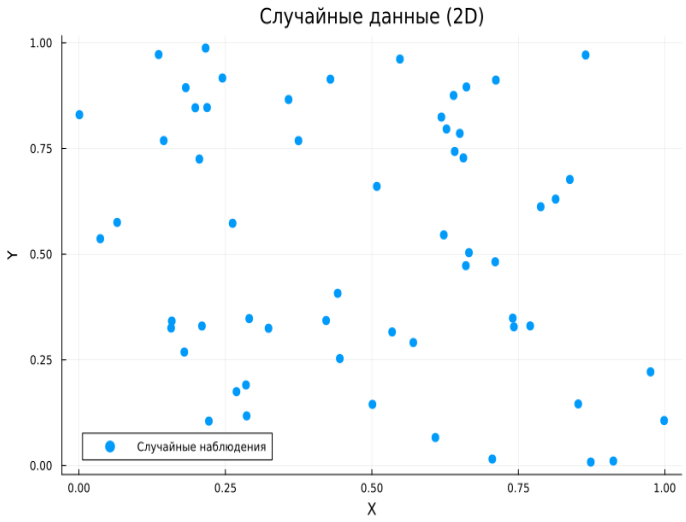


Рисунок 28 — Случайные данные на плоскости

[55]:
```
Random.seed!(8)
x8, y8, z8 = rand(80), rand(80), rand(80)
scatter(x8, y8, z8, label="Случайные наблюдения",
        title="Случайные данные (3D)", xlabel="X", ylabel="Y", zlabel="Z",
        markersize=4, markerstrokewidth=0)
```

[55]:

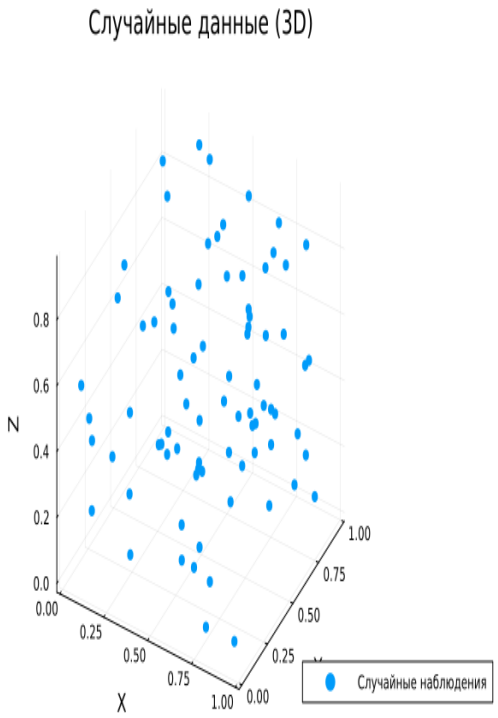


Рисунок 29 — Случайные данные в трёхмерном пространстве

2.6. Анимация синусоиды

В девятом задании создана последовательность графиков, в которой диапазон построенной синусоиды постепенно увеличивается. Последовательность кадров объединяется в файл sine_animation.gif. Ниже показан пример одного кадра.

[57]:
```
x9 = collect(range(0, 2π, length=300))
anim_sin = @animate for last_index in 2:5:length(x9)
    plot(x9[1:last_index], sin.(x9[1:last_index]),
         xlims=(0,2π), ylims=(-1.1,1.1),
         title="Построение синусоиды",
         xlabel="x", ylabel="sin(x)", label="sin(x)", color=:blue)
end
gif(anim_sin, "sine_animation.gif", fps=20)
println("Сохранено: sine_animation.gif")
```

[57]:

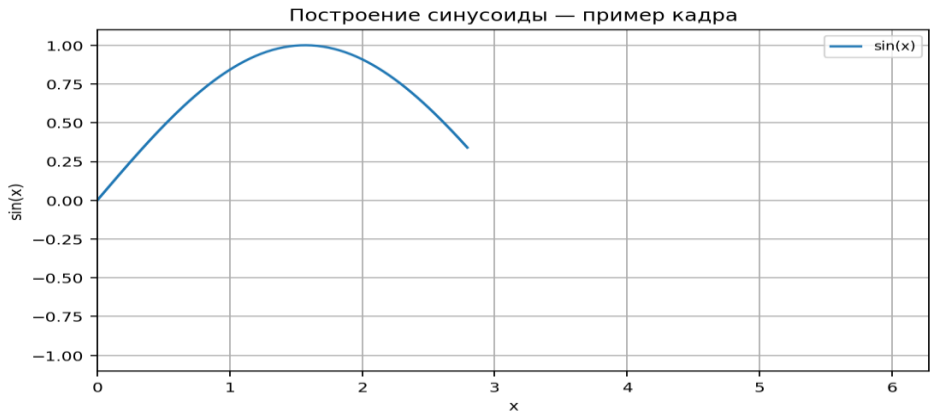


Рисунок 30 — Пример кадра анимации построения синусоиды

2.7. Гипоциклоиды и эпициклоиды

В десятом задании построены анимированные гипоциклоиды для двух целых и двух рациональных значений параметра k: 2, 3, 3/2 и 5/2. Для каждого значения в ноутбуке создаётся отдельный GIF-файл. На рисунке приведены формы получающихся траекторий.

[59]:

```
function hypo_xy(k; r=1.0, n=240)
    q = denominator(rationalize(k))
    t = range(0, 2π*q, length=n)
    x = r*(k-1).*cos.(t) .+ r.*cos.((k-1).*t)
    y = r*(k-1).*sin.(t) .- r.*sin.((k-1).*t)
    return collect(t), x, y
end

function epi_xy(k; r=1.0, n=240)
    q = denominator(rationalize(k))
    t = range(0, 2π*q, length=n)
    x = r*(k+1).*cos.(t) .- r.*cos.((k+1).*t)
    y = r*(k+1).*sin.(t) .- r.*sin.((k+1).*t)
    return collect(t), x, y
end

function animate_curve(kind, k, filename; n=180)
    t, x, y = kind == :hypo ? hypo_xy(k; n=n) : epi_xy(k; n=n)
    title_text = kind == :hypo ? "Гипоциклоида, k=$k" : "Эпициклоида, k=$k"
    anim = @animate for i in 2:length(t)
        plot(x[1:i], y[1:i], aspect_ratio=:equal,
             xlims=(-2.2*(k+1), 2.2*(k+1)),
             ylims=(-2.2*(k+1), 2.2*(k+1)),
             title=title_text, xlabel="x", ylabel="y",
             label="Траектория", color=:blue, linewidth=2)
        scatter!([x[i]], [y[i]], label="Движущаяся точка",
                 color=:red, markersize=4)
    end
    gif(anim, filename, fps=20)
    println("Сохранено: ", filename)
end

k_values = [2, 3, 3//2, 5//2]
for k in k_values
    animate_curve(:hypo, float(k), "hypocycloid_k_$(replace(string(k), "/" => "_")).gif")
end
```

[59]:

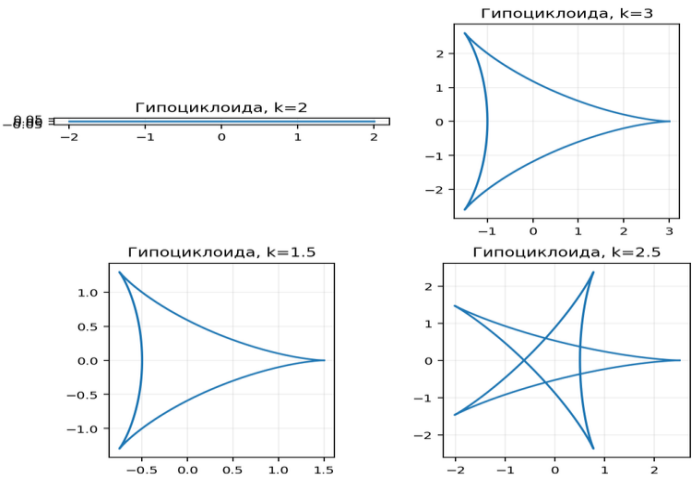


Рисунок 31 — Гипоциклоиды при разных значениях k

В одиннадцатом задании аналогично построены анимированные эпициклоиды для значений k=2, 3, 3/2 и 5/2. Для каждого случая сохраняется отдельная анимация.

[60]:
```
for k in k_values
    animate_curve(:epi, float(k), "epicycloid_k_$(replace(string(k), "/" => "_")).gif")
end
```

[60]:

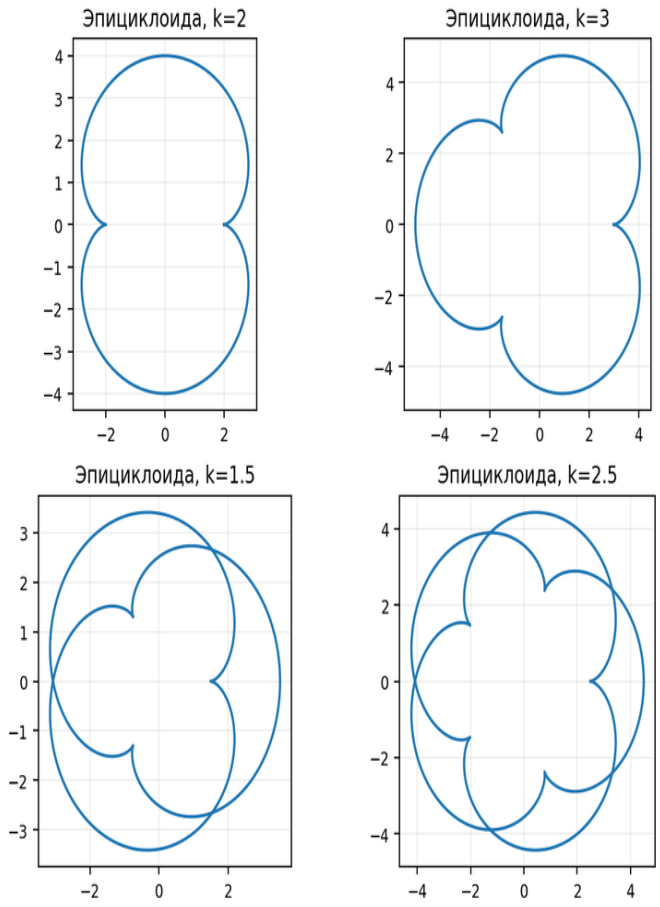


Рисунок 32 — Эпициклоиды при разных значениях k

Выводы

В ходе лабораторной работы освоены основные возможности библиотеки Plots.jl для построения и оформления графиков на языке Julia. Повторены примеры с линейными и точечными графиками, аппроксимацией, двумя осями ординат, полярными и параметрическими кривыми, поверхностями, контурными линиями, векторными полями, гистограммами и графиками погрешностей.

Выполнены задания для самостоятельной работы: построены различные типы графиков функции синуса, применены разные стили линий, оформлен график функции $\pi x^2 \ln(x)$, показаны четыре варианта визуализации функции $x^3 - 3x$, сравнены способы отображения двух функций на одной и двух осях Y. Также созданы графики случайных и экспериментальных данных и подготовлены анимации синусоиды, гипоциклоид и эпициклоид.

Таким образом, получены практические навыки визуализации математических функций и данных в JupyterLab с использованием Julia и Plots.jl.